Evaluating the Robustness of Neural Language Models to Input Perturbations

Milad Moradi, Matthias Samwald
Institute for Artificial Intelligence
Medical University of Vienna, Austria
{milad.moradivastegani, matthias.samwald}@meduniwien.ac.at

Abstract

High-performance neural language models have obtained state-of-the-art results on a wide range of Natural Language Processing (NLP) tasks. However, results for common benchmark datasets often do not reflect model reliability and robustness when applied to noisy, real-world data. In this study, we design and implement various types of character-level and word-level perturbation methods to simulate realistic scenarios in which input texts may be slightly noisy or different from the data distribution on which NLP systems were trained. Conducting comprehensive experiments on different NLP tasks, we investigate the ability of high-performance language models such as BERT, XLNet, RoBERTa, and ELMo in handling different types of input perturbations. The results suggest that language models are sensitive to input perturbations and their performance can decrease even when small changes are introduced. We highlight that models need to be further improved and that current benchmarks are not reflecting model robustness well. We argue that evaluations on perturbed inputs should routinely complement widely-used benchmarks in order to yield a more realistic understanding of NLP systems' robustness.

1 Introduction

High-performance deep neural language models such as BERT (Devlin et al., 2018), XLNet (Z. Yang et al., 2019), and GPT-2 (Radford et al., 2019) have brought breakthroughs to a wide range of Natural Language Processing (NLP) tasks including text classification, sentiment analysis, textual entailment, natural language inference, machine translation, and question answering. Their immense ability in capturing various linguistic properties has led these state-of-the-art language models to master different NLP tasks, even surpassing human accuracy on some benchmarks such as SQuAD[1].

However, recent studies have revealed that there is a gap between performing well on benchmarks and actually working under real-world situations (Belinkov and Bisk, 2018; Ribeiro et al., 2020). Even a well-trained, high-performance deep language model can be sensitive to negligible changes in the input that cause the model to make erroneous decisions (M. Sun et al., 2018). This raises serious concerns regarding the robustness/reliability of neural language models utilized in real-world applications. The terms 'robustness' and 'reliability' refer to the ability of a system to perform consistently well in situations where changes to input should not cause a change in the system's output, or the system is expected to properly reflect the change and produce a correct outcome.

Applying automatic or human-controlled perturbations to textual inputs has been shown to be effective for evaluating the robustness of NLP systems, investigating their vulnerabilities, and finding their bugs. Recently, CheckList (Ribeiro et al., 2020) provided a framework for behavioral testing of NLP systems inspired by black-box

[1] https://rajpurkar.github.io/SQuAD-explorer/

testing in software engineering. CheckList enabled generating new (perturbed) test samples through abstracting different test types aimed at testing linguistic capabilities. Other studies focused on evaluating robustness to perturbed inputs for machine translation (Belinkov and Bisk, 2018; Niu et al., 2020), perturbation sensitivity analysis for detecting unintended model biases (Prabhakaran et al., 2019), or robustness to adversarial perturbations (Alshemali and Kalita, 2020; Ebrahimi et al., 2018; Liang et al., 2018). However, a comprehensive methodology for evaluating the performance of NLP models under real-world conditions is still missing.

In a realistic scenario, the input text may contain typos and misspellings that should not cause any changes in the NLP system's outcome. Minor grammatical errors may appear in the text, but the semantics is still preserved, therefore, the NLP system is expected to treat the input as it was error-free. Some deliberate or unintentional changes may modify the semantics, and the NLP model is expected to reflect the changes in the outcome. These are only few examples of natural noise in text data that NLP systems should have the ability to properly deal with.

In this paper, we design and implement a wide range of character-level and word-level systematic perturbations to textual inputs in order to simulate different types of noise that a NLP system may face in real-world use cases. Conducting extensive experiments on various NLP tasks, we investigated the ability of four neural language models, i.e. BERT, RoBERTa, XLNet, and ELMo, in handling slightly perturbed inputs. The results reveal that the neural models are unstable to small changes that can be easily handled by humans, e.g. misspellings, missing words, repeated words, synonyms, etc. The systematic input perturbations can expose the vulnerabilities of NLP systems and bring more insights into how high-performance models behave when they encounter noisy yet understandable inputs. This study suggests that the performance of NLP models should not be overestimated by only relying on accuracy scores obtained on benchmark datasets.

Similar to CheckList, our perturbation framework treats NLP systems as black-boxes. This facilitates comparison of different models, without needing to know the model structure and internals. CheckList focuses on testing linguistic capabilities of NLP systems, e.g. handling coreferences, identifying named entities, semantic role labeling, and vocabulary. On the other hand, our perturbation methods aim at evaluating the robustness of NLP systems to noisy inputs. Our input perturbation framework can act as a complement to the CheckList testing methodology.

In CheckList, many test types rely on creating synthetic samples from scratch by the user, which is a time-consuming task, and needs much creativity and effort. Moreover, synthetic samples may suffer from low coverage (Ribeiro et al., 2020). However, most of the perturbation methods introduced in this paper do not need human intervention; they can automatically generate perturbed samples that still preserve the semantics and are sufficiently meaningful to users.

Some types of perturbation utilized in this work were already tested in previous work on adversarial attacks on NLP systems (Zeng et al., 2020; Zhang et al., 2020). However, adversarial perturbations are considered worst-case scenarios that do not occur frequently in real-world situations, representing a very specific type of noise (Fawzi et al., 2016). In order to generate effective adversarial examples, most attack methods need to have access to the NLP model structure, internal weights, and hyperparameters, which may not be possible in every testing scenario (Zhang et al., 2020). Furthermore, adversarial perturbations should not be perceived by humans (Liang et al., 2018). This is a serious challenge, since even small changes to a text may be easily recognized by the user.

To the best of our knowledge, this paper is the first study that presents empirical results achieved with a comprehensive set of non-adversarial perturbation methods for testing robustness of NLP systems on non-synthetic text. An important contribution of this work is to evaluate the robustness of several high-performance language models on various NLP tasks using different types of character-level and word-level input perturbations. Moreover, to ascertain the usefulness of the perturbations (i.e. how effectively they can be used to automatically generate meaningful and understandable perturbed samples), we conducted an extensive user study.

2 NLP tasks

In our experiments, we used five datasets covering five different NLP tasks. Table 1 summarizes some statistics of the datasets. A short description of the datasets is given in the following.

TREC (Li and Roth, 2002) is a Text Classification (TC) dataset containing more than 6,000 questions and 50 different class labels that specify the type of questions.

Stanford Sentiment Treebank (SST) (Socher et al., 2013) is a Sentiment Analysis (SA) dataset containing more than 11,000 movie reviews from 'Rotten Tomatoes'. Every review is classified into one of the five classes: *very positive*, *positive*, *neutral*, *negative*, and *very negative*.

CoNLL-2003[2] is a Named Entity Recognition (NER) dataset containing news stories from the Reuters corpus with more than 200K tokens annotated as *Person*, *Organization*, *Location*, *Miscellaneous*, or *Other*.

STS benchmark (Cer et al., 2017) is a Semantic Similarity (SS) dataset comprising of more than 8K text pairs extracted from image captions, news headlines, and user forums. Each pair of sentences is assigned a similarity score between 0 and 5.

WikiQA (WQA) (Y. Yang et al., 2015) is a Question Answering (QA) dataset composed of more than 3,000 questions and 29,000 sentences as answers extracted from Wikipedia.

3 Language models

In our experiments, we utilized four neural language models shown to be effective in learning bidirectional contexts and obtained state-of-the-art results during recent years:

BERT (Devlin et al., 2018) is composed of deep encoder transformer layers and uses two pretraining objectives, i.e. masked language modelling and next sentence prediction. We used the $\text{BERT}_{\text{LARGE}}$ architecture (along with the *cased* model) containing 24 transformer layers, 1024 hidden units per layer, 16 attention heads per hidden unit, and 340 million parameters.

RoBERTa (Liu et al., 2019) uses a model architecture similar to BERT, but adopts an optimized pretraining approach. It was pretrained on more data, with bigger batch sizes and longer sequences than BERT. Furthermore, the next sentence prediction objective was removed and a dynamic masking strategy replaced the basic masking method. We used $\text{RoBERTa}_{\text{LARGE}}$ that further optimizes the same model as $\text{BERT}_{\text{LARGE}}$.

XLNet (Z. Yang et al., 2019) utilizes decoder transformers and adopts a permutation language modelling approach along with generalized autoregressive pretraining. We used the $\text{XLNet}_{\text{LARGE}}$ model, with the same architecture hyperparameters and model size as $\text{BERT}_{\text{LARGE}}$.

Dataset	Task	Train	Dev	Test	Eval. Measure
TREC	TC	5,000	452	500	Micro F1-score
SST	SA	8,544	1,101	2,210	Accuracy
CoNLL	NER	14,041	3,250	3,453	F1-score
STS	SS	5,749	1,500	1,379	Pearson
WQA	QA	2,117	296	630	F1-score

Table 1: The main statistics of the datasets used to conduct the perturbation experiments.

ELMo (Peters et al., 2018) is a contextualized word representation method that utilizes character convolutions along with shallow concatenation of backward and forward LSTMs to implement bidirectional language modeling. We used the original ELMo model composed of two highway layers with an LSTM hidden size of 4096, output size of 512, and a total parameters of 93.6 million. The contextualized embeddings computed by ELMo were fed into a dense layer containing 128 hidden units followed by an output layer with a softmax activation in the TC and QA tasks, a linear activation in the SA and SS tasks, and CRF layer with a linear activation in the NER task.

We retrieved the pretrained models, fine-tuned them separately on each downstream task using the training and development sets, and tested them on the test sets. We utilized the Huggingface transformers (Wolf et al., 2020) and FARM [3] libraries to implement the transformer-based models. A complete list of hyperparameter values is presented in Appendix A.

4 Perturbation methods

We designed and implemented various character-level and word-level perturbation methods that simulate different types of noise an NLP system may encounter in real-world situations. The perturbations can be produced for every dataset regardless of the underlying language model or NLP system being tested. Table 2 presents an example for every perturbation method. The perturbation methods were implemented in Python using the NLTK library. The source code is available at `https://github.com/mmoradi-iut/NLP-perturbation`.

[2] `https://github.com/synalp/NER/tree/master/corpus/CoNLL-2003`

[3] `https://github.com/deepset-ai/FARM`